

조인(JOIN)

등가 조인과 비등가 조인

학습목표

1. 여러 테이블의 데이터를 검색하는 조인을 학습한다.
2. 조인 조건을 이해한다.
3. 조인 조건에 따른 등가조인과 비등가 조인에 대해 학습한다.
4. 조인 문법을 학습한다.
5. Cartesian product에 대해 학습한다.

SELECT문의 기능

프로젝션(Projection)

선택(Selection)

조인(Join)

조인이란? : 등가 조인

```
SQL> SELECT * FROM emp;
```

ENO	ENAME	SEX	JOB	MGR	HDATE	SAL	COMM	DNO
0001	안영희	여	경영		91/01/01	4800	0	01
0201	안영숙	여	지원	0001	91/02/01	3900	2000	01
0202	손하늘	여	지원	0001	91/12/01	3510	980	01
0301	이승철	남	회계	0001	92/02/01	3400	0	02
0302	박선경	여	회계	0301	91/03/02	3300	0	02
1001	문시현	남	모델링	0201	91/02/01	4500	520	10
1002	김주란	여	모델링	0201	92/03/03	4100	330	20
1003	양선호	남	모델링	0201	95/02/21	4300		30
2001	남궁연호	남	개발	0202	93/12/13	3950	200	10
.....								

```
SQL> SELECT * FROM dept;
```

DNO	DNAME	LOC
01	총무	서울
02	회계	서울
10	ERP	서울
20	ISP	부산
30	ITEA	광주
40	CRM	대전
50	POS	

emp.dno = dept.dno

조인이란? : 비등가 조인

```
SQL> SELECT * FROM emp;
```

ENO	ENAME	SEX	JOB	MGR	HDATE	SAL	COMM	DNO
0001	안영희	여	경영		91/01/01	4800	0	01
0201	안영숙	여	지원	0001	91/02/01	3900	2000	01
0202	손하늘	여	지원	0001	91/12/01	3510	980	01
0301	이승철	남	회계	0001	92/02/01	3400	0	02
0302	박선경	여	회계	0301	91/03/02	3300	0	02
1001	문시현	남	모델링	0201	91/02/01	4500	520	10
1002	김주란	여	모델링	0201	92/03/03	4100	330	20
1003	양선호	남	모델링	0201	95/02/21	4300		30
2001	남궁연호	남	개발	0202	93/12/13	3950	200	10
.....								

```
SQL> SELECT * FROM salgrade;
```

GRADE	LOSAL	HISAL
1	4000	9999
2	3000	3999
3	2000	2999
4	1000	1999
5	0	999

emp.sal BETWEEN salgrade.losal AND salgrade.hisal
(emp.sal >= salgrade.losal AND emp.sal <= salgrade.hisal)

조인

```
SELECT 테이블.컬럼, .....  
FROM 테이블, 테이블, .....  
WHERE 조인_조건  
AND 일반_조건  
ORDER BY 컬럼 .....;
```

- 조인 조건 : 테이블간의 관계 수식
- 조인 문장에서 기술된 모든 테이블은 반드시 조인 조건으로 기술됨.
- 동일한 이름의 컬럼이 여러 테이블에 존재하는 경우 컬럼명 앞에 반드시 테이블명이 필요함.
- 테이블 개수가 n개일 때 조인 조건은 항상 (n-1)개 이상.
- 등가 조인(Equi-join) : 조인 조건이 등호('=')
- 비 등가 조인(Non equi-join) : 조인 조건이 부등호

등가 조인

- 각 사원의 근무 부서와 근무지를 검색한다.

```
SQL> SELECT dept.dno 부서번호, dname 부서, loc 근무처,  
2          eno 사번, ename 이름  
3 FROM dept, emp  
4 WHERE dept.dno=emp.dno;
```

```
SQL> SELECT d.dno 부서번호, dname 부서, loc 근무처,  
2          eno 사번, ename 이름  
3 FROM dept d, emp e  
4 WHERE d.dno=e.dno;
```

- 테이블에 별명은 SQL문을 간결하게 하지만 가독성에 문제가 있을 수 있다.
- 별명이 부여된 경우 테이블명은 사용할 수 없다.
 - . 'ORA-00904: ... 부적합한 식별자' 에러 발생

비등가 조인

- 각 사원의 급여 등급을 검색한다.

```
SQL> SELECT eno, ename, sal, grade  
2  FROM emp e, salgrade s  
3  WHERE sal BETWEEN losal AND hisal;
```

- 총무 부서 사원의 급여를 10% 인상한 경우 사원의 급여등급을 검색한다.

```
SQL> SELECT d.dno, dname, eno, ename, sal, sal*1.1, grade  
2  FROM dept d, emp e, salgrade s  
3  WHERE d.dno = e.dno AND sal*1.1 BETWEEN losal AND hisal  
4  AND dname = '총무';
```


잘못된 조인 조건

- 조인 조건 누락

```
SQL> SELECT d.dno 부서번호, dname 부서, loc 근무처,  
2          eno 사번, ename 이름  
3 FROM dept d, emp e;
```

- Cartesian product : 조인 조건이 부족하거나 빠진 경우 검색결과
 - . 각 테이블의 행의 수를 곱한 개수만큼 행이 검색된다.
 - . 특별한 경우 cross join으로 사용된다.
- 조인 조건은 n-1개 이상이다.

실습

1. 송강 교수가 강의하는 과목을 검색한다.
2. 과목명에 화학이 포함된 과목을 강의하는 교수의 명단을 검색한다.
3. 학점이 2학점인 과목과 이를 강의하는 교수를 검색한다.
4. 화학과 교수가 강의하는 과목을 검색한다.
5. 화학과 1학년 학생의 기말고사 성적을 검색한다.
6. 일반화학 과목의 기말고사 점수를 검색한다.
7. 화학과 1학년 학생의 일반화학 기말 고사 점수를 검색한다.
8. 화학과 1학년 학생이 수강하는 과목을 검색한다.
9. 일반화학 과목에서 평가 점수가 A인 학생의 명단을 검색한다.
10. 송강 교수의 과목을 수강하는 학생의 기말고사 점수를 성적 순서로 검색한다.
11. 화학과 1학년 학생의 기말고사 성적을 학점(A,B,C,D,F)으로 검색한다.
12. 송강 교수가 강의하는 과목에서 평가 점수가 A인 학생의 명단을 과목명과 함께 검색한다.
13. 화학과 1학년 학생에게 강의하는 교수의 명단을 검색한다.

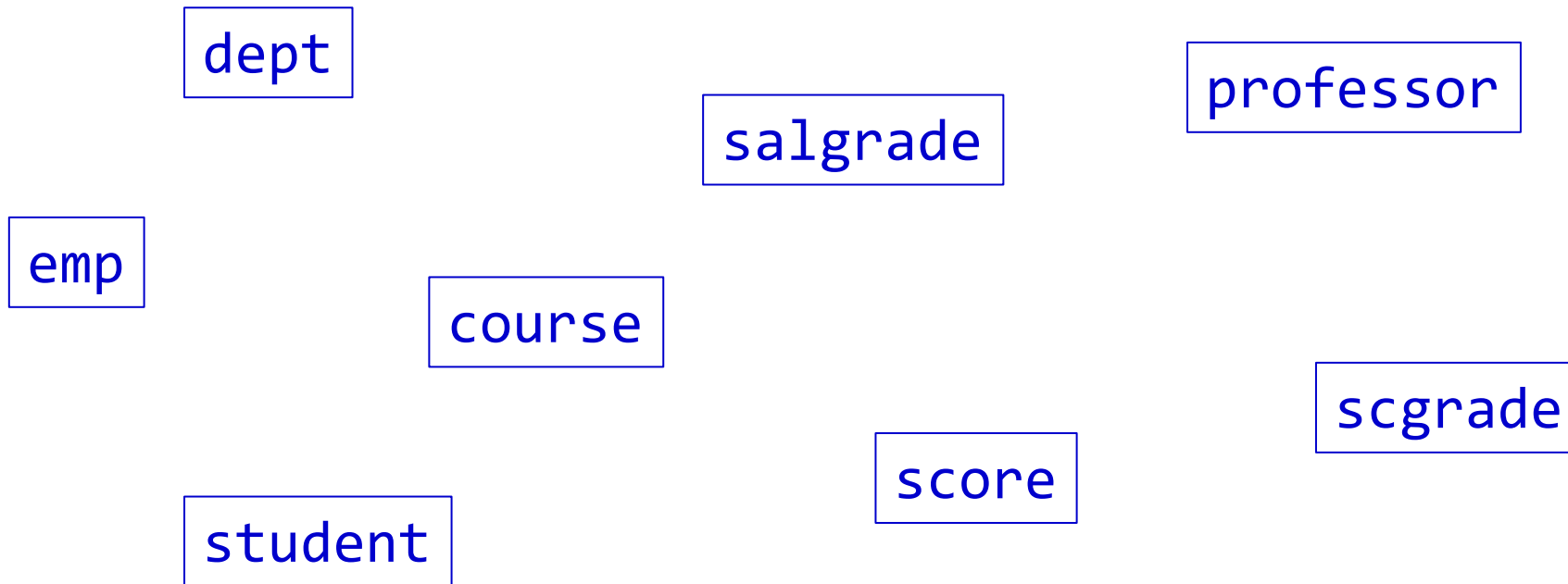
조인문을 쉽게

학습목표

1. 테이블간 관계를 확인한다.
2. 관계를 조인 조건으로 표현한다.
3. 조인문을 만드는 과정을 학습니다.

조인문 작성의 어려움 1.

- 조인문을 작성하기 어려운 이유
 - 테이블 구조와 관계를 숙지하지 못함!!!



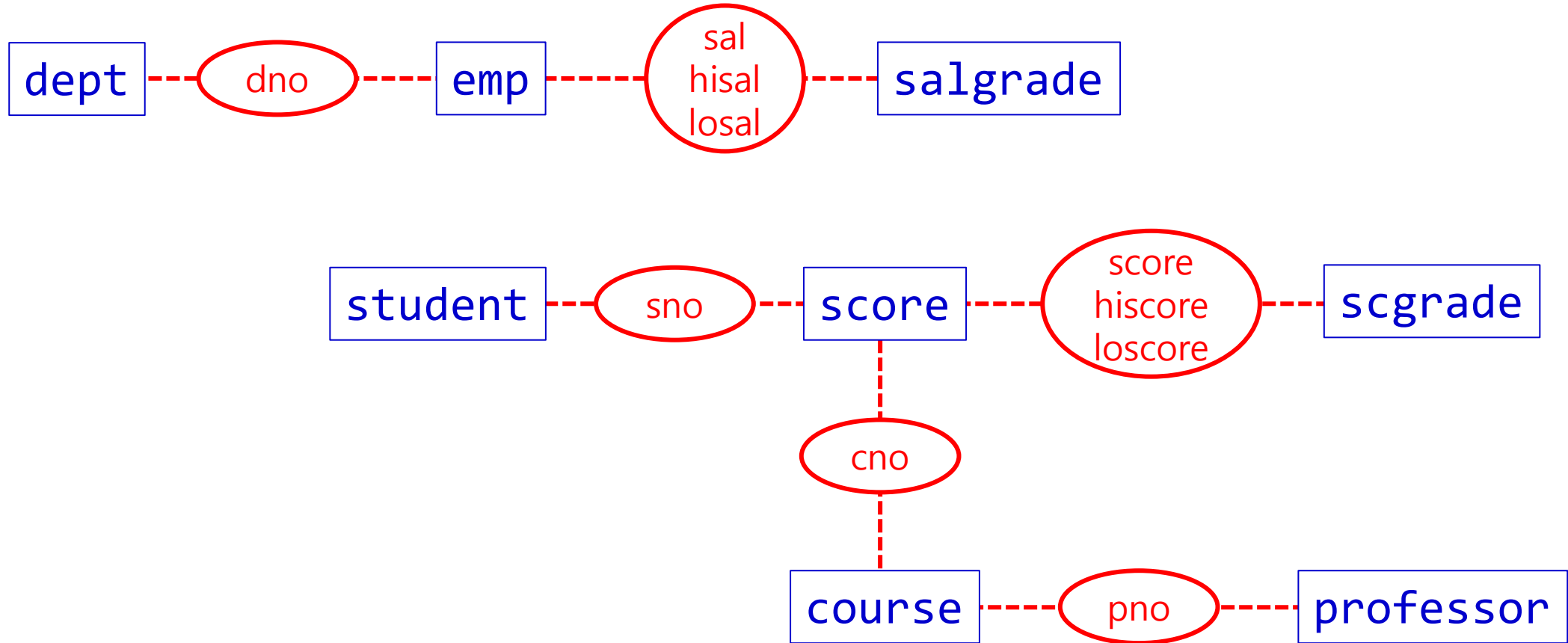
테이블 상세 구조

emp(사원)		dept(부서)		salgrade(급여등급)		scgrade(점수등급)	
컬럼	내용	컬럼	내용	컬럼	내용	컬럼	내용
eno	사번	dno	부서번호	grade	등급	grade	등급
ename	이름	dname	부서명	losal	하한값	loscore	하한값
sex	성별	loc	지역	hisal	상한값	hiscore	상한값
job	업무						
mgr	사수사번						
hdate	입사일						
sal	급여						
comm	보너스						
dno	부서번호						

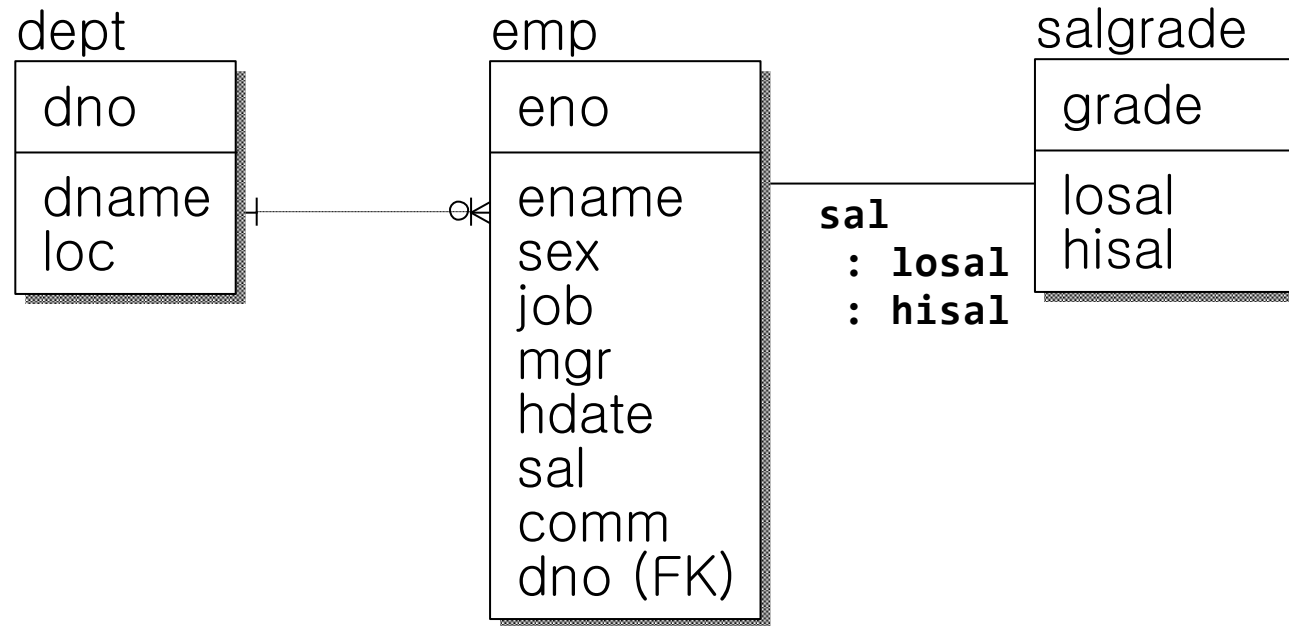
student(학생)		professor(교수)		course(과목)		score(기말고사점수)	
컬럼	내용	컬럼	내용	컬럼	내용	컬럼	내용
sno	학번	pno	교수번호	cno	과목번호	sno	학번
sname	이름	pname	교수이름	cname	과목명	cno	과목번호
sex	성별	section	소속학과	st_num	학점수	result	점수
syear	학년	orders	직위	pno	교수번호		
major	학과	hiredate	부임일				
avr	평점(4.0)						

테이블 표현과 관계

- RDB이 모든 테이블은 관계가 있다.



ERD를 이용한 테이블 표현



조인문 작성의 어려움 2

- 프로세스화 된 조인문 작성 과정을 구축하기 어려움.

- 조인문 작성과정(지문기준)

Step 1 검색대상과 조건을 추출

- SELECT, WHERE, ORDER BY절 구성

Step 2 SELECT절과 FROM 절에서 검색 테이블 추출

Step 3 테이블간 관계 확인

- 관계 확인을 통해 테이블이 추가될 수 있다.

Step 4 조인 조건 기술

Step 5 문장을 완성한다.

- SELECT 절에 내용이 추가된다.

조인문 작성 실습 1

- 지문

화학과 일반 화학 기말고사 점수를 검색한다.

Step 1. 검색대상과 조건을 추출

검색 대상

- 기말고사 점수 : result

조건

- 화학과 학생 : major = '화학'
- 일반 화학 과목 : cname = '일반 화학'

```
SELECT result
FROM
WHERE major = '화학' AND cname = '일반화학';
```

Step 2 SELECT절과 FROM 절에서 검색 테이블 추출

- result : score
- major : student
- cname : course

```
SELECT result
FROM student, course, score
WHERE major = '화학' AND cname = '일반화학';
```

조인문 작성 실습 1

Step 3 테이블간에 관계 확인

- FROM 절에 기술된 테이블간에 관계를 확인한다.
- 관계가 연결이 끊어지면 안된다.
 - 관계를 위해 FROM절에 테이블을 추가할 수 있다.



Step 4 조인 조건 기술

- FROM 절 테이블간 관계

```
SELECT result
FROM student s, course c, score r
WHERE s.sno = r.sno AND r.cno = c.cno
AND major = '화학' AND cname = '일반화학';
```

Step 5 문장을 완성한다.

- 문장의 의미를 갖도록.

```
SELECT major, cname, s.sno, sname, result
FROM student s, course c, score r
WHERE s.sno = r.sno AND r.cno = c.cno
AND major = '화학' AND cname = '일반화학';
```

조인문 작성 실습 2

- 지문 : 화학과 1학년 학생에게 강의하는 교수의 명단을 검색한다.

Step 1. 검색대상과 조건을 추출

검색 대상

- 교수 명단: pno, pname

조건

- 화학과 학생 : major = '화학'
- 1 학년: syear = 1

```
SELECT pno, pname  
FROM  
WHERE major = '화학' AND syear = 1;
```

Step 2 SELECT절과 FROM 절에서 검색 테이블 추출

- pno, pname : professor
- major : student
- syear : student

```
SELECT pno, pname  
FROM student s, professor p  
WHERE major = '화학' AND syear = 1;
```

조인문 작성 실습 2

Step 3 테이블간에 관계 확인

- Student와 professor는 직접 관계가 없다. : course와 score 가 필요하다.



Step 4 조인 조건 기술

```
SELECT p.pno, pname
FROM student s, score r, course c, professor p
WHERE s.sno=r.sno AND r.cno=c.cno AND c.pno=p.pno
AND major = '화학' AND syear = 1;
```

Step 5 문장을 완성한다.

```
SELECT DISTINCT p.pno, pname 화학과_학생에게_강의하는_교수명단
FROM student s, score r, course c, professor p
WHERE s.sno=r.sno AND r.cno=c.cno AND c.pno=p.pno
AND major = '화학' AND syear = 1;
```

실습

1. 송강 교수가 강의하는 과목을 검색한다.
2. 과목명에 화학이 포함된 과목을 강의하는 교수의 명단을 검색한다.
3. 학점이 2학점인 과목과 이를 강의하는 교수를 검색한다.
4. 화학과 교수가 강의하는 과목을 검색한다.
5. 화학과 1학년 학생의 기말고사 성적을 검색한다.
6. 일반화학 과목의 기말고사 점수를 검색한다.
7. 화학과 1학년 학생의 일반화학 기말 고사 점수를 검색한다.
8. 화학과 1학년 학생이 수강하는 과목을 검색한다.
9. 일반화학 과목에서 평가 점수가 A인 학생의 명단을 검색한다.
10. 송강 교수의 과목을 수강하는 학생의 기말고사 점수를 성적 순서로 검색한다.
11. 화학과 1학년 학생의 기말고사 성적을 학점(A,B,C,D,F)으로 검색한다.
12. 송강 교수가 강의하는 과목에서 평가 점수가 A인 학생의 명단을 과목명과 함께 검색한다.
13. 화학과 1학년 학생에게 강의하는 교수의 명단을 검색한다.

자기 참조 조인과 외부 조인

학습목표

1. 자기참조 조인을 학습한다.
2. 외부 조인을 학습한다.

자기 참조 조인(Self join)

```
SELECT  별명1.컬럼1, ... 별명2.컬럼1, ...  
FROM    테이블 별명1, 테이블 별명2, ...  
WHERE   조인_조건  
AND     일반_조건;
```

- 동일 테이블을 자기 참조에 의해 조인한다.
- 별명1과 별명2는 동일 테이블에 대한 별명이지만 각각 별도의 테이블처럼 사용한다.
- 하나의 테이블에 별명을 다르게 해서 마치 여러 개의 테이블간 조인하는 것처럼 문장을 구현한다.

Self join

- 각 사원과 이를 관리하는 사원의 정보를 검색한다.

```
SQL> SELECT e1.eno, e1.ename, e1.mgr, e2.eno, e2.ename  
2 FROM emp e1, emp e2  
3 WHERE e1.mgr = e2.eno;
```

- e1, e2 테이블간 등가 조인
- e1 : 사원 테이블, e2 : 관리자 테이블

외부(Outer) 조인

```
SELECT 컬럼, 컬럼, ...  
FROM 테이블1, 테이블2, ...  
WHERE 조인_조건(+)  
AND 일반_조건
```

- 조인 조건에 일치하는 않는 데이터를 출력해 준다.
- 조인 결과물을 해석할 때 오해를 방지하기 위한 방법이다.
 - . 결과의 무결성을 보장한다.
- '+' 기호는 데이터가 부족한 쪽에 기술한다.

외부 조인 결과를 비교한다.

- 각 부서별로 사원의 명단을 검색한다.

```
SQL> SELECT d.dno 부서번호, dname 부서명, ename 사원명  
2   FROM dept d, emp e  
3   WHERE d.dno = e.dno  
4   ORDER BY 1;
```

```
SQL> SELECT d.dno 부서번호, dname 부서명, ename 사원명  
2   FROM dept d, emp e  
3   WHERE d.dno = e.dno(+) (+)  
4   ORDER BY 1;
```

실습

1. 학생 중에 동명이인을 검색한다.
2. 모든 직원의 명단과 이를 관리하는 직원의 명단을 검색한다.
3. 직원 중에 자신의 관리자 보다 급여가 높은 사람의 급여 정보를 관리자 급여 정보와 같이 검색 한다.
4. 전체 교수 명단과 교수가 담당하는 과목의 이름을 학과 순으로 검색한다.
5. 모든 과목 목록과 각 과목별 담당 교수를 검색한다.

단일 행 서브쿼리

학습목표

1. 서브 쿼리의 개념을 이해한다.
2. 단일 행 서브 쿼리를 학습한다.
3. 단일 행 서브 쿼리의 문제점을 학습한다.

서브 쿼리

- SQL 문에 완전한 SELECT문이 포함됨
 - 원래 SQL문 : 메인 쿼리
 - 내부 SELECT 문: 서브 쿼리
- 서브 쿼리의 종류
 - 단일 행 서브 쿼리
 - . 서브 쿼리가 하나의 컬럼에서 하나의 행만 검색한다.
 - . 즉 데이터 하나를 검색한다.
 - 다중 행 서브 쿼리
 - . 서브 쿼리가 하나의 컬럼에서 여러 개의 행을 검색한다.
 - 다중 열 서브 쿼리
 - . 서브 쿼리가 여러 컬럼에서 행을 검색한다.
- 서브 쿼리는 응용 프로그램으로 대체 가능하지만 성능이나 비용 면에서 월등하다.

단일 행 서브 쿼리

```
SELECT 컬럼 ...  
FROM 테이블  
WHERE 컬럼 단일_행_연산자 (SELECT 문장 : Sub query문)  
.....;
```

- 단일 행 연산자가 사용됨으로 반드시 서브 쿼리의 결과 값은 한 개만 검색되어야 한다.
- 서브 쿼리는 반드시 괄호로 묶는다.
- 서브 쿼리는 메인 쿼리 실행 전에 실행된다.
- 서브 쿼리의 검색된 결과 값은 메인 쿼리에 사용된다.
- 단일 행 연산자 오른쪽에 기술한다.
 - . =, <, >, <=, >=, !=
- WHERE절에 기술된 열의 숫자와 타입은 SELECT절과 1:1 대응 관계가 되어야 한다.

단일 행 서브 쿼리

- 남궁연호보다 급여를 많이 받는 사원을 검색한다.
- 문시현과 부서가 다르고 동일한 업무를 하는 사원의 정보를 검색한다.

```
SQL> SELECT eno 사번, ename 이름
2 FROM emp
3 WHERE sal > (SELECT sal
4              FROM emp
5              WHERE ename = '남궁연호');
```

```
SQL> SELECT eno 사번, ename 이름, dno 부서번호, job 업무
2 FROM emp
3 WHERE dno != (SELECT dno FROM emp WHERE ename = '문시현')
4 AND job = (SELECT job FROM emp WHERE ename = '문시현');
```

단일 행 서브 쿼리의 문제

- 김선유보다 급여를 많이 받는 사원을 검색한다.

```
SQL> SELECT eno 사번, ename 이름  
2   FROM emp  
3   WHERE sal > (SELECT sal  
4                 FROM emp  
5                 WHERE ename = '김선유');
```

- 'ORA-01427: 단일 행 하위 질의에 2개 이상의 ~'
 - 서브 쿼리에서 2개 이상의 행이 검색된 경우
- 예측이 어려운 단일 행 서브쿼리 수정방법
 - '=' 연산자는 'IN' 연산자로 바꾼다.
 - 부등호 ('<', '>', '<=', '>=')는 any, all 다중 행 연산자를 추가한다.
 - Max(), Min()과 같은 그룹 함수를 사용 한다.

서브 쿼리와 조인

- 부산에서 근무하는 사원의 정보 검색

```
SQL> SELECT eno||' '||ename 부산_근무_사원
2  FROM emp
3  WHERE dno = (SELECT dno
4                FROM dept
5                WHERE loc = '부산');
```

```
SQL> SELECT eno||' '||ename 부산_근무_사원
2  FROM emp e, dept d
3  WHERE e.dno = d.dno
4  AND loc = '부산';
```

실습

1. 김혁윤 보다 평점이 우수한 학생의 학번과 이름을 검색한다.
2. 권현와 동일한 학년 학생 중에 평점이 강은혜와 동일한 학생을 검색한다.
3. 이학수학 과목과 동일한 학점수인 과목을 검색한다.
4. 타 학과에 송강 교수와 동일한 지위의 교수 명단을 검색한다.
5. 제갈민의 입사일 보다 늦게 부임한 교수의 명단을 검색한다.
6. 강태용보다 일반 화학 과목의 학점이 더 낮은 학생의 명단을 학점과 검색한다.
7. 양선호의 입사일보다 빨리 부임한 교수의 명단을 검색한다.

다중 행, 다중 열 서브 쿼리

학습목표

1. 다중 행 서브 쿼리를 학습한다.
2. ANY, ALL 연산자 특징에 대해 학습한다.
3. 다중 열 서브 쿼리를 학습한다.

다중 행 서브 쿼리

```
SELECT 컬럼 ...  
FROM 테이블  
WHERE 컬럼 다중_행_연산자 (SELECT 문장 : Sub query문)  
.....;
```

- 서브 쿼리에 여러 행이 검색되는 경우 사용한다.
- 다중 행 연산자를 이용한다.
- 다중 행 연산자의 종류
 - . IN : 나열된(검색된) 값 중에 하나만 일치하면 된다.
 - . ANY : 나열된(검색된) 값 일부와 일치하면 된다.
 - . ALL : 나열된(검색된) 값 전체와 일치해야 한다.

IN 연산자를 이용한 다중 행 서브 쿼리

- 화학과 교수의 부임일과 동일한 날 입사한 사원의 명단을 검색한다.
- 20번 부서원과 동일한 관리자로 부터 관리 받는 사원을 검색한다.

```
SQL> SELECT eno 사번, ename 이름  
2 FROM emp  
3 WHERE hdate IN (SELECT hiredate  
4 FROM professor  
5 WHERE section = '화학');
```

```
SQL> SELECT * FROM emp  
2 WHERE mgr IN (SELECT mgr FROM emp  
3 WHERE dno = '20')  
4 AND dno != 20; ← 매우 많은 실수!!
```

ANY, ALL을 이용한 다중 행 서브 쿼리

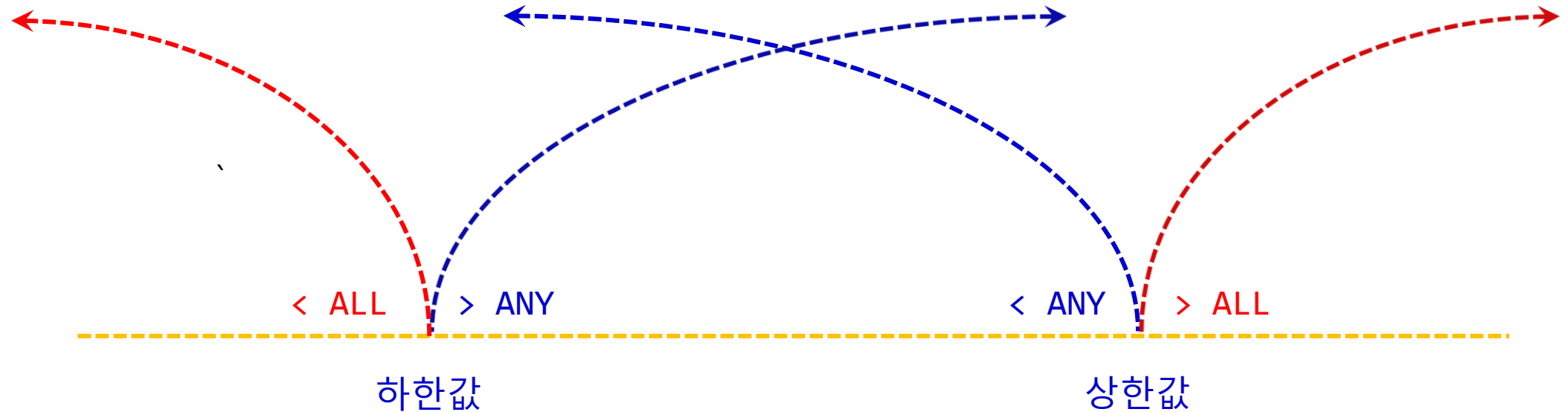
- 10번 부서원들 보다 급여가 낮은 사원을 검색한다.

```
SQL> SELECT eno 사번, ename 이름, dno 부서번호  
2   FROM emp  
3   WHERE sal < ALL (SELECT sal  
4                     FROM emp  
5                     WHERE dno = '10');
```

- 10번 부서원들의 급여 중 가장 낮은 값보다 낮은 급여를 받는 사원 검색
- 10번 부서원이 검색되지 않는다.
- 서브 쿼리를 그룹함수를 이용해서 수정할 수 있다.

```
(SELECT MIN(sal)  
FROM emp  
WHERE dno = '10');
```

ANY, ALL을 이용한 다중 행 서브 쿼리



- * 컬럼 > ALL → 컬럼 > MAX() : 가장 큰 값보다 크다
- * 컬럼 < ALL → 컬럼 < MIN() : 가장 작은 값보다 작다.
- * 컬럼 > ANY → 컬럼 > MIN() : 가장 작은 값보다 크다.
- * 컬럼 < ANY → 컬럼 < MAX() : 가장 큰 값보다 작다

다중 열 서브 쿼리

```
SELECT 컬럼 ...  
FROM 테이블  
WHERE (컬럼1, 컬럼2, ...) IN (SELECT 문장 : Sub query문)  
.....;
```

- 서브 쿼리의 SELECT문에 여러 개의 컬럼을 검색한다.
- 여러 개의 컬럼을 검색하는 서브 쿼리 문을 이용할 때는 반드시 비교 대상 컬럼과 1:1 대응돼야 한다.
- 다중 열 서브 쿼리에서 서브 쿼리의 검색 결과가 단지 하나의 행이라면 '=' 연산자 사용이 가능하지만 되도록 'IN' 연산자를 이용한다.

다중 열 서브쿼리

- 손하늘과 동일 관리자의 관리를 받으면서 업무도 같은 사원을 검색한다.

```
SQL> SELECT eno 사번, ename 이름, mgr 관리자, job 업무
2   FROM emp
3   WHERE (mgr, job) IN (SELECT mgr, job
4                        FROM emp
5                        WHERE ename = '손하늘')
6   AND ename != '손하늘';
```

```
SQL> SELECT eno 사번, ename 이름, mgr 관리자, job 업무
2   FROM emp
3   WHERE mgr = (SELECT mgr FROM emp WHERE ename = '손하늘')
4   AND job = (SELECT job FROM emp WHERE ename = '손하늘')
5   AND ename != '손하늘';
```

다중 열 서브쿼리

- 김선유와 부서 및 업무가 동일한 같은 사원을 검색한다.

```
SQL> SELECT eno 사번, ename 이름, dno 부서번호, job 업무
2   FROM emp
3   WHERE (dno, job) IN (SELECT dno, job
4                        FROM emp
5                        WHERE ename = '김선유')
6   AND ename != '김선유';
```

```
SQL> SELECT eno 사번, ename 이름, dno 부서번호, job 업무
2   FROM emp
3   WHERE dno IN (SELECT dno FROM emp WHERE ename = '김선유')
4   AND job IN (SELECT job FROM emp WHERE ename = '김선유')
5   AND ename != '김선유';
```

실습

1. 화학과 학생과 평점이 동일한 학생들을 검색한다.
2. 화학과 교수와 부임일이 같은 직원을 검색한다.
3. 화학과 학생과 같은 학년에서 평점이 동일한 학생들을 검색한다.
4. 10번 부서 직원들보다 년봉을 많이 받는 직원을 검색한다.
5. 10번 부서 직원들과 업무와 성별이 동시에 일치하는 직원을 검색한다.

집합 연산자

학습목표

1. 집합 연산자에 대해서 학습한다.
2. UNION과 UNION ALL의 차이점에 대해서 학습한다.
3. 집합 연산자를 이용한 SELECT 문을 학습한다.

집합 연산자

```
SELECT ...  
[UNION ALL | UNION | INTERSECT | MINUS]  
SELECT ...
```

- 두 SELECT 문의 결과를 집합 연산한다.
- 검색 결과의 헤더는 앞쪽 SELECT문에 의해 결정된다.
- 두 SELECT 문의 컬럼 개수와 데이터 타입이 일치해야 한다.

연산자	의미	결과
UNION	합집합	검색된 결과의 합으로 중복이 제거된다.
UNION ALL		검색된 결과의 합으로 중복을 허용한다.
INTERSECT	교집합	양쪽 모두에 포함된 결과만 검색한다.
MINUS	차집합	첫 번째 검색 결과에서 두 번째 검색 결과를 제외한 나머지만 검색한다.

집합 연산자를 이용한 검색

- 2000년 이후에 입사한 사원과 부임한 교수의 명단을 부임일 순으로 검색한다.

```
SQL> SELECT pno 번호, pname 이름, hiredate 입사일_부임일
2  FROM professor
3  WHERE hiredate >= '2000/01/01'
4  UNION ALL
5  SELECT eno, ename, hdate
6  FROM emp
7  WHERE hdate >= '2000/01/01'
8  ORDER BY 3;
```

- 집합 연산자를 이용한 출력에서 헤더는 첫 번째 SELECT문에 의해 결정된다.
- ORDER BY절은 SELECT문 끝에 추가한다.
 - . pno나 eno가 아니라 반드시 번호를 이용해서 정렬한다.

집합 연산자를 이용한 검색

- 화학, 물리학과 학생들 중에 학점이 3.0 이상인 학생을 검색한다.
- 제갈씨 성을 가진 사원 중에 지원 업무를 하지 않는 사원을 검색한다.

```
SQL> SELECT sno 학번, sname 이름, major 학과, avr 학점 FROM student
2  WHERE major IN ('화학','물리')
3  INTERSECT
4  SELECT sno, sname, major, avr FROM student
5  WHERE avr >= 3;
```

```
SQL> SELECT eno, ename, job FROM emp
2  WHERE ename LIKE '제갈%'
3  MINUS
4  SELECT eno, ename, job FROM emp
5  WHERE job = '지원';
```

실습

다음 실습은 집합 연산자를 이용해 검색한다.

1. 화학과 학생과 교수를 검색한다. (집합 연산자를 이용한다.)
2. 정교수의 명단과 모델링이 업무인 직원의 이름, 입사일(부임일)을 검색한다.
3. 과목 중에 개설되지 않은 과목을 검색한다.
4. 모든 학생의 평점을 검색한다. 단 화학과 학생만 4.5 환산평점으로 검색한다.

단일 행 함수 문자 함수

학습목표

1. 단일 행 함수에 대해 학습한다.
2. 문자 함수에 대해 학습한다.
3. 문자 치환 함수에 대해 학습한다.

단일 행 함수

- 단일 행 함수를 이용하면 검색된 데이터에 대해서 각 행별로 연산된 결과 값을 얻을 수 있으며 중첩이 가능하다.
- 단일 행 함수는 응용 프로그램의 오류를 줄이고 개발 시간을 단축 할 수 있다.
- 인수로는 컬럼 명, 수식, 변수, 상수 등을 사용할 수 있다.
함수명(인수1, 인수2, ...)
- 단일 행 함수는 SELECT절이나 WHERE절 또는 HAVING 절 등 값이 필요한 거의 모든 구문에 사용이 가능하다.
- 단일 행 함수의 종류
 - 문자 함수, 숫자 함수, 날짜 함수, 변환 함수, 일반 함수 ...

문자 함수

함수명	기능	함수명	기능
LOWER	문자열을 소문자로 변환한다. LOWER(문자열)	INSTR	문자의 위치 반환 INSTR(문자열, 검색문자, 시작위치, 횟수)
	LOWER('ORACLE') → oracle		INSTR('ORACLE', 'A') → 3
UPPER	문자열을 대문자로 변환한다. UPPER(문자열)	TRIM	접두(접미) 글자를 잘라낸다 TRIM([leading trailing <u>both</u>] 제외문자 FROM 문자열)
	UPPER('oracle') → ORACLE		TRIM('o' FROM 'oracle') → racle
INITCAP	첫 번째 문자는 대문자로, 나머지는 소문자로 변환한다. INITCAP(문자열)	LENGTH LENGTHB	문자열의 길이나 BYTE를 반환한다. LENGTH(문자열) LENGTHB(문자열)
	INITCAP('oracle') → Oracle		LENGTH('디비') → 2 LENGTHB('디비') → 4
SUBSTR	문자열내 지정된 위치의 문자열을 검색한다. SUBSTR(문자열, STR, CNT)	LPAD, RPAD	지정된 문자열의 길만큼 빈 부분에 문자를 채운다. LPAD(문자열, 출력폭, 채움문자)
	SUBSTR('oracle', 1, 2) → or		LPAD('20000', 10, '*') → *****20000

문자 함수를 이용한 검색

- 부서명과 문자열의 길이를 출력한다.
- SUBSTR 함수를 이용해서 사원 이름의 일부 내용만을 검색한다.
- 다양한 방법으로 'a'가 나오는 위치를 출력한다.

```
SQL> SELECT dname, LENGTH(dname), LENGTHB(dname)
2 FROM dept;
```

```
SQL> SELECT ename, SUBSTR(ename,2),
2          SUBSTR(ename,-2),
3          SUBSTR(ename,1,2),
4          SUBSTR(ename,-2,2)
5 FROM emp;
```

```
SQL> SELECT INSTR('database','a'),
2          INSTR('database','a',3),
3          INSTR('database','a',1,3)
4 FROM dual;
```

문자 함수를 이용한 검색

- 부서명의 마지막 글자를 제외하고 검색한다.

```
SQL> SELECT dname, SUBSTR(dname, 1, LENGTH(dname)-1) dname  
2 FROM dept;
```

- SQL에서 지원하는 여러 함수들은 필요에 따라 위와 같이 중첩 사용된다.

- 이름이 네 글자인 사원의 이름을 검색한다.

```
SQL> SELECT eno, ename  
2 FROM emp  
3 WHERE LENGTH(ename) = 4;
```

문자 함수 : 문자 치환 함수

단일행	기능
TRANSLATE	문자 단위로 치환된 값을 반환한다. TRANSLATE(문자열, 검색문자, 치환문자)
	TRANSLATE('oracle', 'a','#') → or#cle
REPLACE	문자열 단위로 치환된 값을 반환한다. REPLACE(문자열, 검색문자열, 치환문자열)
	REPLACE('oracle', 'or', '##') → ##acle

문자 함수 : 문자 치환 함수

- 형식이 비슷한 TRANSLATE와 REPLACE를 동일한 치환을 통해 비교해 보자.
 - 원문의 'Wo'를 '--'로 변환 한다.

```
SQL> SELECT TRANSLATE('World of Warcraft', 'Wo', '--') Translate,  
2          REPLACE('World of Warcraft', 'Wo', '--') Replace  
3 FROM dual;
```

- TRANSLATE는 각각의 글자를 지정된 문자로 변환하는 것에 주의한다.

실습

다음 실습은 반드시 문자 함수를 이용해서 검색한다.

1. '강' 씨 성을 가진 학생의 이름을 검색한다.
2. 교수의 지위를 한글자로 검색한다.(ex. 조교수 → 조)
3. 모든 일반 과목 명칭을 기초 과목으로 변경해서 검색한다.
(ex. 일반화학 → 기초화학)
4. 만일 입력 실수로 student 테이블의 sname 컬럼에 데이터가 입력될 때 문자열 마지막에 공백이 추가 되었다면 검색할 때 이를 제외하고 검색하는 SELECT문을 작성한다.
5. 직원의 년봉을 10자리로 검색한다. 단 공백은 임의의 채움 문자로 채워 넣는다.
6. 학생의 이름을 검색한다. 단 성이 '심'인 학생은 성을 '사마' 로 바꾸어 검색한다.

단일 행 함수 숫자 함수와 날짜 연산

학습목표

1. 다양한 숫자 함수를 학습한다.
2. 날짜 데이터에 대해서 학습한다.
3. 날짜 연산을 학습한다.
4. 날짜 검색에서 주의할 사항을 학습한다.

숫자 함수

함수명	기능	함수명	기능
ROUND	반올림해서 소수 이하 n 자리까지 출력한다. ROUND(m, n)	CEIL	m 보다 큰 가장 작은 정수를 검색한다. CEIL(m)
	ROUND(123.4567, 3) → 123.457		CEIL(2.34) → 3
TRUNC	소수 이하 n 자리 미만을 절삭한다. TRUNC(m, n)	FLOOR	m 보다 작은 가장 큰 정수를 검색한다. FLOOR(m)
	TRUNC(123.4567, 3) → 123.456		FLOOR(2.34) → 2
MOD	m을 n으로 나눈 나머지를 계산한다. MOD(m, n)	SQRT	제곱근을 검색한다. SQRT(m)
	MOD(10, 4) → 2		SQRT(9) → 3
POWER	m의 n승을 계산한다. POWER(m, n)	SIGN	음수 : -1, 양수 : 1, 0 이면 0을 검색한다. SIGN(m)
	POWER(2, 4) → 16		SIGN(-3) → -1

숫자 함수의 이용

- 다양한 숫자 함수의 이용한 검색 결과를 확인한다.

```
SQL> SELECT ROUND(98.765), TRUNC(98.765),  
2          ROUND(98.765,2), TRUNC(98.765,2)  
3 FROM dual;
```

```
SQL> SELECT MOD(19,3), MOD(-19,3)  
2 FROM dual;
```

```
SQL> SELECT CEIL(3.5), FLOOR(3.5)  
2 FROM dual;
```

```
QL> SELECT POWER(13,4), SQRT(16)  
2 FROM dual;
```

날짜 연산

- 날짜 데이터

- 문자처럼 표기하지만 숫자처럼 계산된다.
- 6 byte 크기로 저장된다.
- 년 월 일 시 분 초 (YYYY MM DD HH24 MI SS)를 저장한다.
- 출력은 터미널의 출력 식에 따른다.
- 날짜관련 검색에서는 보이지 않는 부분에 주의한다.
- 예약어 처럼 쓰이는 sysdate 함수를 이용해서 시스템의 현재 날짜를 검색한다.

- 날짜 연산

연산	결과	해석
날짜 + m	날짜	m 일 이후 날짜
날짜 - m	날짜	m 일 이전 날짜
날짜 - 날짜	숫자	두 날짜 간에 기간(일 수)

- 시간 : $m/24$

날짜 검색

- 현재 날짜를 다양한 포맷으로 검색한다.

```
SQL> SELECT sysdate FROM dual;
```

```
SQL> ALTER SESSION SET nls_date_format = 'YYYY/MM/DD';
```

```
SQL> SELECT sysdate FROM dual;
```

```
SQL> ALTER SESSION SET nls_date_format = 'YYYY/MM/DD:HH24:MI:SS';
```

```
SQL> SELECT sysdate FROM dual;
```

- Windows 기본 날짜 출력 형식 : 'YY/MM/DD'
- UNIX 기본 날짜 출력 형식 : 'DD-MON-YY'
- nls_date_format은 세션 레벨에서 설정한다.
- glogin.sql 파일을 이용한다.
- 응용 프로그램에서 날짜 관련 검색은 반드시 **TO_DATE**나 **TO_CHAR**를 이용한다.

날짜 연산

- 오늘 날짜, 입사 일자, 입사일로부터 오늘까지 기간, 입사일 이후 100째 되는날 등을 검색하고 날짜 연산의 결과를 살펴보자.

```
SQL> SELECT eno, ename, sysdate 오늘, hdate 입사일,  
2          TRUNC(sysdate)-TRUNC(hdate)+1 근무일, hdate+99 "100일"  
3  FROM emp;
```

- TRUNC는 시분초를 버린다.
- 날짜 연산은 ROUND하는 경우가 거의 없고 TRUNC하는 것이 대부분이다.

단일 행 함수 날짜 함수

학습목표

1. 날짜 함수에 대해서 학습한다.
2. 날짜 함수 사용시 유의 사항에 대해서 학습한다.
3. 날짜 포맷에 대해서 학습한다.

날짜 함수

함수명	기능		함수명	기능
ROUND	형식에 따라 반올림한다. ROUND(날짜, 형식)	YYYY 년	LAST_DAY	날짜를 포함한 달의 마지막 날짜를 검색한다. LAST_DAY(날짜)
	ROUND(sysdate, 'DD') → '2021/09/23'	MM 월		LAST_DAY('2013/09/24') → 2013/09/30
TRUNC	형식에 따라 절삭한다. TRUNC(날짜, 형식)	DD 일	ADD_MONTHS	날짜에 n 달을 더한 날짜를 검색한다. ADD_MONTHS(날짜, n)
	TRUNC(sysdate, 'YYYY') → '2021/01/01'	HH24 시		ADD_MONTHS('2011/07/01', 23) → 2013/06/01
MONTHS_BETWEEN	두 날짜 간의 기간을 월 수로 계산한다. 날짜1 > 날짜2 : 양수. MONTHS_BETWEEN(날짜1, 날짜2)	HH 시	NEXT_DAY	날짜 이후 지정된 요일에 해당하는 날짜를 검색한다. - 일요일, SUNDAY, 1 모두 동일하다. NEXT_DAY(날짜, 요일)
	MONTHS_BETWEEN('2013/09/01', '2013/01/01') → 8	MI 분		NEXT_DAY('2013/07/14', '일요일') → 2013/07/21
		SS 초		

날짜 함수의 이용

- 날짜를 다양한 단위에서 반올림하거나 절삭해 본다.

```
SQL> ALTER SESSION SET nls_date_format = 'YYYY/MM/DD:HH24:MI:SS';
```

```
SQL> SELECT sysdate,  
2          ROUND(sysdate, 'YYYY') 년,  
3          ROUND(sysdate, 'MM') 월,  
4          ROUND(sysdate, 'DD') 일  
5 FROM dual;
```

```
SQL> SELECT sysdate,  
2          TRUNC(sysdate, 'YY') 년,  
3          TRUNC(sysdate, 'MM') 월,  
4          TRUNC(sysdate, 'DD') 일  
5 FROM dual;
```

날짜 함수의 이용

- 문시현이 오늘까지 일한 일수를 검색한다.

```
SQL> SELECT ename, TRUNC(sysdate)-TRUNC(hdate)+1 DAY  
2 FROM emp  
3 WHERE ename='문시현';
```

- 20번 부서 직원들이 현재까지 근무한 개월 수를 검색한다.

```
SQL> SELECT eno, ename, TRUNC(MONTHS_BETWEEN(sysdate,hdate)) 근무_개월  
2 FROM emp  
3 WHERE dno = '20';
```

- 20번 부서원들이 입사한 이후 첫 번째 일요일을 검색한다.

```
SQL> SELECT eno, ename, hdate, NEXT_DAY(hdate,'일요일') Sunday  
2 FROM emp  
3 WHERE dno = '20';
```

날짜 함수의 이용

- 20번 부서원들이 입사 100일째 되는 날과 10년째 되는 날을 검색한다.

```
SQL> SELECT eno, ename, hdate 입사일,  
2          hdate+99 "100일", ADD_MONTHS(hdate,120) "10년"  
3 FROM emp  
4 WHERE dno = '20';
```

- 20번 부서원들의 입사한 달의 마지막 날짜와 입사한 달에 근무 일수를 검색한다.

```
SQL> SELECT eno, ename, hdate, LAST_DAY(hdate) 마지막날,  
2          LAST_DAY(TRUNC(hdate))-TRUNC(hdate)+1 "입사한 달 근무일수"  
3 FROM emp  
4 WHERE dno = '20';
```

실습

1. 교수들이 부임한 달에 근무한 일수는 몇 일인지 검색한다.
2. 교수들의 오늘까지 근무한 주가 몇 주인지 검색한다.
3. 1991년에서 1995년 사이에 부임한 교수를 검색한다.
4. 사원들이 일한 날짜에 대해서만 급여를 받는다면 급여가 현재와 동일하다는 조건에서 입사한 달에 급여는 얼마나 지급되었을지 검색한다.

단일 행 함수 변환 함수

학습목표

1. 변환 함수기능에 대해 학습한다.
2. 변환 함수 용도에 대해 학습한다.
3. 형식 문자에 대해 학습한다.

날짜 시간 형식

	형식	의미		형식	의미
년	YYYY	네 자리 년도 : 2021, 1999	시	HH24	01 ~ 24 , 24시간 표현
	YY	두 자리 년도 : 99-2099		HH	01 ~ 12, 12시간 표현
	RR	두 자리 년도 : 99-1999			
월	MM	숫자 : 01, 02	분	MI	##, 분
	MONTH	문자 : MARCH, APRIL, 3월, 4월	초	SS	##, 초
	MON	약자 : MAR, APR, 3월, 4월		SSSSSS	하루를 초로 환산 (0 ~ 86399)
일	DD	숫자 : 01, 02	오전 오후	AM/PM	오전, 오후 형식으로 의미 없음.
	DAY	요일 : SUNDAY, 일요일			
	DY	약자 요일 : MON, 월			

숫자 형식

형식	의미
9	숫자의 출력 폭을 지정한다.
0	선행 0 을 표기한다.
\$	화폐 표기(달러)
L	화폐 표기(각 국가의 언어 셋 설정에 따라 다르다.)
,	쉼표 위치 지정(보통은 세자리마다 지정한다.)
.	소수점 위치를 지정한다.
MI	음수의 - 기호 표시를 오른쪽에 표기한다.
EEEE	실수 표현기법

변환 함수

함수명	기능
TO_CHAR	날짜나 숫자를 지정된 형식의 문자로 변환한다. - 출력 형식을 지정한다. TO_CHAR(날짜, 형식), TO_CHAR(숫자, 형식)
TO_DATE	문자를 지정된 형식의 날짜로 해석한다. TO_DATE(문자, 형식)
TO_NUMBER	문자를 숫자로 해석한다. 대부분의 경우 오라클의 자동 형변환에 의해 SQL 구문 작성시에는 사용빈도가 매우 낮다.

- TO_CHAR : 데이터를 출력하거나 전달할 때 서식에 맞추어 가공하는 기능을 제공한다. 이를 통해 응용 프로그램의 부담을 줄이고 가독성과 생산성을 높일 수 있다.
- TO_DATE : 사용자가 입력하거나 전달 받은 문자열을 시스템이 정확한 형식에 따라 날짜로 해석하도록 하는 함수로 날짜 데이터에 반드시 사용한다.

TO_CHAR를 이용한 날짜 검색

- 오늘 날짜를 다양한 형식으로 검색한다.

```
SQL> SELECT TO_CHAR(sysdate, 'YYYY/MM/DD') 날짜,  
2          TO_CHAR(sysdate, 'YYYY/MM/DD:HH24:MI:SS') 날짜,  
3          TO_CHAR(sysdate, 'YY/MM/DD:HH:MI:SS AM') 날짜  
4 FROM dual;
```

```
SQL> SELECT TO_CHAR(sysdate, 'DAY Mon YY') 날짜 FROM DUAL;
```

← 9월, Sep(September) 검색

```
SQL> SELECT ename||' 사원의 입사일은 '||  
2          TO_CHAR(hdate, 'YYYY"년 "MM"월 "DD"일 입니다."'') 입사일  
3 FROM emp;
```

TO_CHAR를 이용한 숫자 검색

- 숫자를 다양한 형식으로 검색한다.

```
SQL> SELECT TO_CHAR(12345.678, '999,999.99999') num FROM dual;
```

```
SQL> SELECT TO_CHAR(12345.678, '099,999.999') num FROM dual;
```

```
SQL> SELECT TO_CHAR(12345.678, '99,999.9') num FROM dual;
```

```
SQL> SELECT TO_CHAR(12345.678, '9,999.9') num FROM dual;
```

```
SQL> SELECT TO_CHAR(1234, '$999,999') num FROM dual;
```

```
SQL> SELECT TO_CHAR(1234, 'L999,999') num FROM dual;
```

```
SQL> SELECT TO_CHAR(123456789, '9.999EEEE') num FROM dual;
```

```
SQL> SELECT TO_CHAR(-1234, '999,999MI') num FROM dual;
```

TO_CHAR를 이용한 검색

- 10번 부서 사원의 보너스가 급여의 몇 퍼센트 인지 검색한다.

```
SQL> SELECT eno 사번, ename 이름,  
2          TO_CHAR(NVL(comm,0)/(sal*12)*100, '90.99') || '%' 급여_비율  
3 FROM emp  
4 WHERE dno='10';
```

```
SQL> SELECT eno 사번, ename 이름,  
2          ROUND(NVL(comm,0)/(sal*12)*100,2) || '%' 급여_비율  
3 FROM emp  
4 WHERE dno='10';
```

TO_DATE 함수를 이용한 검색

- 1992년 이전에 입사한 사원의 정보를 검색한다.

```
SQL> SELECT eno 사번, ename 이름, hdate 입사일  
2 FROM emp  
3 WHERE hdate < TO_DATE('19920101', 'YYYYMMDD');
```

```
SQL> SELECT eno 사번, ename 이름, hdate 입사일  
2 FROM emp  
3 WHERE hdate <= TO_DATE('19911231:235959', 'YYYYMMDD:HH24MISS');
```

실습

1. 학생의 평균 평점을 다음 형식에 따라 소수점 이하 두 자리까지 검색한다.
 - 'XXX 학생의 평균 평점은 x.xx입니다.'
2. 교수의 부임 일을 다음 형식으로 검색한다.
 - 'XXX 교수의 부임일은 YYYY년 MM월 DD일입니다.'
3. 교수 중에 3월에 부임한 교수의 명단을 검색한다.
4. 화학과 1학년 학생들의 4.5 환산 평점을 다음 형식에 따라 소수점 이하 두 자리까지 검색한다.
 - 'XXX 학생의 4.5 환산 평점은 x.xx입니다.'

그룹 함수와 GROUP BY

학습목표

1. 그룹함수의 종류와 기능에 대해서 학습한다.
2. GROUP BY 절에 대해서 학습한다.
3. 카디널리티에 대해서 학습한다.

그룹 함수

- 여러 행에 컬럼을 기반으로 계산된 값을 제공하는 다중행 함수를 의미한다.

함수	설명
MAX(컬럼)	컬럼 값 중에 최대 값을 반환한다.
MIN(컬럼)	컬럼 값 중에 최소 값을 반환한다.
AVG(컬럼)	평균 값을 반환한다.
COUNT(컬럼 *)	NULL이 아닌 행의 수를 반환한다. *는 모든 행의 수를 반환한다.
SUM(컬럼)	컬럼의 합계를 반환한다.
STDDEV(컬럼)	표준 편차를 반환한다.
VARIANCE(컬럼)	분산을 반환한다.

- 그룹 함수에서 NULL은 무시된다.
- GROUP BY 절 없이 일반 컬럼과 같이 기술 될 수 없다.

그룹 함수를 이용한 검색

- 사원의 평균 급여를 검색한다.
- 월간 지급 급여 총액과 연간 지급 급여 총액을 검색한다.

```
SQL> SELECT AVG(sa1) 평균급여, TO_CHAR(AVG(sa1), '$999,999') 평균급여  
2 FROM emp;
```

```
SQL> SELECT SUM(sa1), TO_CHAR(SUM(sa1), '$999,999') 월간_급여_총액,  
2 SUM(sa1)*12, TO_CHAR(SUM(sa1)*12, '$999,999') 년급여_총액  
3 FROM emp;
```

그룹 함수와 NULL

- 사원에게 지급된 보너스 지급 총액과 보너스 평균을 검색한다.

```
SQL> SELECT SUM(comm) 총액, TO_CHAR(AVG(comm), '$999,999') 보너스_평균  
2 FROM emp;
```

```
SQL> SELECT SUM(comm) 총액,  
2          COUNT(comm) 수령인원, TO_CHAR(AVG(comm), '999,999') 평균,  
3          COUNT(*) 전체인원, TO_CHAR(AVG(NVL(comm,0)), '999,999') 평균  
4 FROM emp;
```

- DB에 저장된 NULL은 무결성에 문제를 유발한다.
- 개발자보다는 설계자나 운영자의 문제일 가능성이 높다.

그룹 함수와 다중 행 연산자.

- 10번 부서원들보다 급여가 높은 사원을 검색한다.

```
SQL> SELECT eno 사번, ename 이름, dno 부서번호  
2 FROM emp  
3 WHERE sal > (SELECT MAX(sal)  
4             FROM emp  
5             WHERE dno = '10');
```

```
SQL> SELECT eno 사번, ename 이름, dno 부서번호  
2 FROM emp  
3 WHERE sal > ALL(SELECT sal  
4                FROM emp  
5                WHERE dno = '10');
```

- ANY나 ALL같은 다중 행 서브 쿼리와 동일하지만 가독성은 훨씬 높다.

GROUP BY

```
SELECT ...  
FROM 테이블 ...  
WHERE 조건 ...  
GROUP BY 컬럼  
ORDER BY 정렬_대상 ... ;
```

- GROUP BY 절은 컬럼의 동일한 값을 바탕으로 소그룹에 대한 통계 정보를 검색한다. SELECT 절에 그룹 함수와 같이 쓰인 일반 컬럼이 GROUP BY 절에 기술되지 않으면 카디널리티(cardinality)가 일치하지 않아 'ORA-00937: not a single-group group function'에러가 발생한다.
- 결과 값이 정렬되길 원한다면 반드시 ORDER BY절을 추가한다.

GROUP BY 절을 이용한 검색

- 업무별 평균 급여, 평균 연봉을 검색한다.
- 부서별 평균 급여, 평균 연봉을 검색한다.

```
SQL> SELECT job 업무, TO_CHAR(AVG(sal), '999,999') 평균_급여,  
2          TO_CHAR(AVG(sal*12+NVL(comm,0)), '999,999') 평균_연봉  
3 FROM emp  
4 GROUP BY job;
```

```
SQL> SELECT d.dno 부서번호, dname 부서, TO_CHAR(AVG(sal), '999,999') 평균_급여,  
2          TO_CHAR(AVG(sal*12+NVL(comm,0)), '999,999') 평균_연봉  
3 FROM dept d, emp e  
4 WHERE d.dno = e.dno  
5 GROUP BY d.dno, dname  
6 ORDER BY d.dno;
```

카디널리티(Cardinality) 와 정렬

- 카디널리티

- 값의 개수(기수)를 의미한다.
- SELECT 문의 각 컬럼은 반드시 카디널리티가 같아야한다.
- '그룹 함수와 같이 검색되는 모든 컬럼은 반드시 GROUP BY절에 기술한다.' 이것이 원칙이다.
 - . GROUP BY 절에 기술되어도 그룹 함수와 같이 검색되지 않는 경우도 있다.
 - . 카디널리티가 일치하는 것이 중요하다.

- 정렬

- GROUP BY 절은 정렬을 수행한다.
- 오라클은 해시 알고리즘을 이용하므로 정렬되지 않는다.
- 정렬된 결과를 위해서는 반드시 ORDER BY 절을 이용한다.

GROUP BY 절을 이용한 검색

- 각 부서별 급여 평균이 가장 높은 값과 낮은 값을 검색한다.

```
SQL> SELECT dno 부서번호,  
2          MAX(AVG(sal)) 최대평균, MIN(AVG(sal)) 최소평균  
3 FROM emp  
4 GROUP BY dno;
```

```
SQL> SELECT MAX(AVG(sal)) 최대평균, MIN(AVG(sal)) 최소평균  
2 FROM emp  
3 GROUP BY dno;
```

- 카디널리티가 맞지 않아 GROUP BY절의 컬럼을 SELECT 할 수 없다.

GROUP BY 절을 이용한 검색

- 각 부서별 최소 급여를 받는 사원의 정보를 검색한다.

```
SQL> SELECT d.dno, dname, eno, ename, sal
2  FROM emp e, dept d
3  WHERE d.dno=e.dno
4  AND (d.dno, sal) IN (SELECT dno, MIN(sal)
5                        FROM emp
6                        GROUP BY dno)
7  ORDER BY d.dno;
```

실습

1. 각 학과별 학생 수를 검색한다.
2. 화학과와 생물학과 학생 4.5 환산 평점의 평균을 각각 검색한다.
3. 부임일이 10년 이상 된 직급별(정교수, 조교수, 부교수) 교수의 수를 검색한다.
4. 과목명에 화학이 포함된 과목의 학점수 총합을 검색한다.
5. 화학과 학생들의 기말고사 성적을 성적순으로 검색한다.
6. 학과별 기말고사 평균을 성적순으로 검색한다.
7. 30번 부서의 업무별 연봉의 평균을 검색한다.
 - 단 출력 양식은 소수이하 두 자리까지 통일된 형식으로 출력한다.
8. 물리학과 학생 중에 학년별로 성적이 가장 우수한 학생의 평점을 검색한다.
9. 학년별로 환산 평점의 평균값을 검색한다.
 - 단 출력 양식은 소수이하 두 자리까지 통일된 양식으로 출력한다.
10. 화학과 1학년 학생 중 평점이 평균 이하인 학생을 검색한다.

그룹 함수와 HAVING

학습목표

1. 그룹된 컬럼에 대해서 학습한다.
2. HAVING절 조건에 대해 학습한다.

HAVING

```
SELECT ...  
FROM 테이블 ...  
WHERE 조건 ...  
GROUP BY 컬럼  
HAVING [그룹조건] ...  
ORDER BY 정렬_대상 ... ;
```

- HAVING : 조건 중에 그룹 함수를 포함하는 조건을 기술한다.
 - . 그룹핑된 조건을 기술한다.
- HAVING절은 그룹된 결과에 대한 조건이므로 가능한 GROUP BY절 뒤에 기술하는 것을 권한다.

GROUP BY절에 따른 그룹 함수 결과 값의 변화

```
SQL> SELECT ROUND(AVG(sal))  
2   FROM emp  
3   GROUP BY dno;
```

```
SQL> SELECT ROUND(AVG(sal))  
2   FROM emp  
3   GROUP BY job;
```

- 동일 SELECT 절(그룹 함수)에서도 GROUP BY 절의 그룹 대상에 따라 결과가 달라진다.
- HAVING절은 그룹된 대상에 대한 조건이다.
 - . SELECT 절과는 무관하다.

HAVING 절을 이용한 그룹 검색

- 부서별 급여 평균이 3천 달러 미만인 부서의 평균 급여를 검색한다.

```
SQL> SELECT dno 부서번호, TO_CHAR(AVG(sal), '$999,999') 평균급여  
2 FROM emp  
3 GROUP BY dno  
4 HAVING AVG(sal) < 3000;
```

- 부서별 급여 평균이 3천 달러 미만인 부서의 최대 급여를 검색한다.

```
SQL> SELECT dno 부서번호, MAX(sal) 최대급여  
2 FROM emp  
3 GROUP BY dno  
4 HAVING AVG(sal) < 3000;
```

HAVING의 조건

- 다음 HAVING절의 조건을 확인한다.

```
SQL> SELECT dno 부서번호, COUNT(*) 인원수  
2 FROM emp  
3 GROUP BY dno  
4 HAVING job != '개발';
```

```
SQL> SELECT dno 부서번호, COUNT(*) 인원수  
2 FROM emp  
3 GROUP BY dno  
4 HAVING dno != '10';
```

- 일반 조건은 WHERE 절에 기술한다.
- 그룹 된 컬럼(GROUP BY 절에 기술된)에 대한 조건은 HAVING절에 기술 할 수 있다.
 - . 권하지 않는다.
 - . 일반 조건 : WHERE
 - . 그룹 함수가 포함된 조건 : HAVING

HAVING 절을 이용한 검색

- 부서 중 가장 급여를 많이 받는 부서를 검색한다.

```
SQL> SELECT d.dno, dname, AVG(sal)
2   FROM dept d, emp e
3   WHERE d.dno = e.dno
4   GROUP BY d.dno, dname
5   HAVING AVG(sal) = (SELECT MAX(AVG(sal))
6                       FROM emp
7                       GROUP BY dno);
```

실습

1. 화학과를 제외하고 학과별로 학생들의 평점 평균을 검색한다.
2. 화학과를 제외한 각 학과별 평균 평점 중에 평점이 2.0 이상인 정보를 검색한다.
3. 기말고사 평균이 60점 이상인 학생의 정보를 검색한다.(학번과 기말고사 평균)
4. 강의 학점수가 3학점 이상인 교수의 정보를 검색한다.(교수번호, 이름과 담당 학점수)
5. 기말고사 성적이 핵 화학과목보다 우수한 과목의 과목명과 담당 교수명 검색한다.
6. 근무 중인 직원이 4명 이상인 부서를 검색한다.
7. 업무별 평균 년봉이 2만불 이상인 업무를 검색한다.
8. 각 학과의 학년별 인원중 인원이 5명 이상인 학년을 검색한다.
9. 인원수가 가장 많은 학과를 검색한다.
10. 학생 중 기말고사 성적이 가장 낮은 학생의 정보를 검색한다.